

Algoritmos

Rocio Anabel Negroni

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

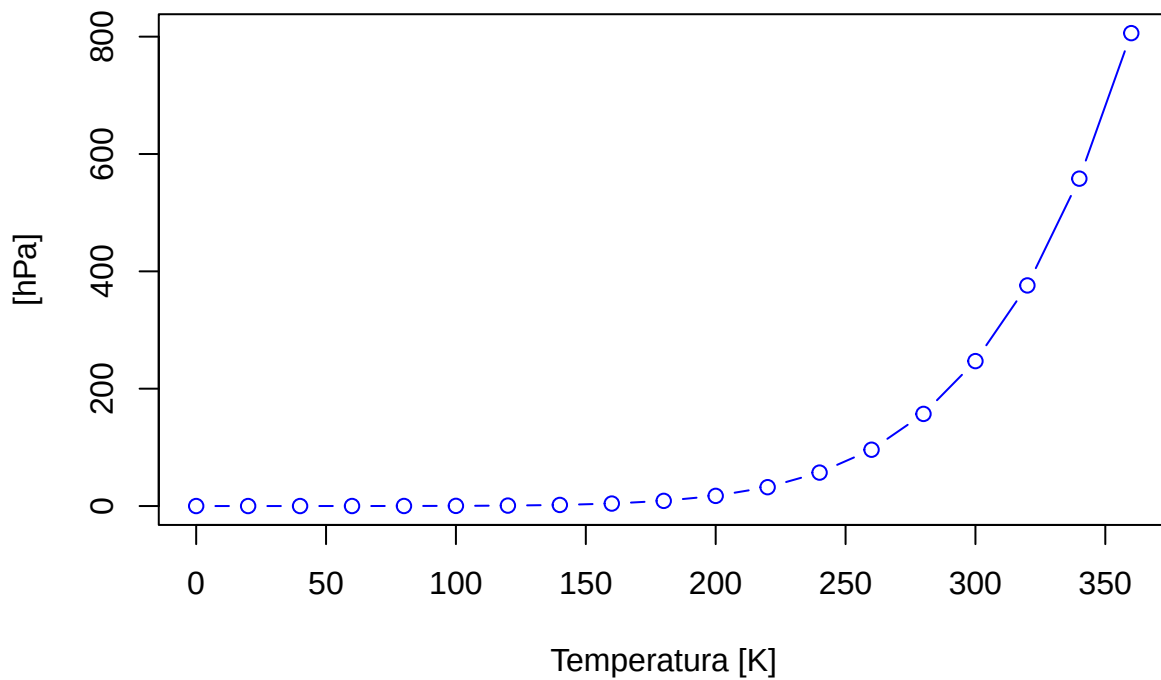
When you click the *Knit* button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a matlab y trabaja generalmente con variables que pueden ser matrices. Para escribir un código oprimimos la tecla +C en verde y R.

```
plot(pressure,main="Presión del gas ideal",ylab="Presión  
[hPa]",xlab="Temperatura [K]",type="b", col="blue")
```

Presión del gas ideal



```
a=38  
a=40  
b=60
```

```
c=b-a
c

## [1] 20

Y=40
alpha<-30
```

Uso de data sets o base de datos internos de R

Usando el comando `data` en la consola obtenemos un listado de distintos datos ya cargados en la base de datos de R. Estos datos están dados en tablas/matrices de las cuales podemos elegir columnas específicas si escribimos signo pesos después del comando de información específico. Por ejemplo `cars$speed`. Nos saltan todas la velocidades que antes la encontrabamos en forma de columna en la matriz de `cars`.

Conclusion

El uso de Posit Cloud representa una solución práctica y accesible para iniciarse en el análisis de datos con R, especialmente en entornos académicos y colaborativos. Su facilidad de acceso, sin necesidad de instalaciones complejas, favorece el aprendizaje y el trabajo en equipo desde cualquier lugar. No obstante, al depender de internet y contar con recursos limitados, puede no ser la opción más adecuada para proyectos de gran escala o que requieran alto rendimiento computacional. En consecuencia, se posiciona como una herramienta ideal para la formación, la experimentación y el desarrollo de proyectos intermedios, mientras que los trabajos más exigentes pueden beneficiarse de un entorno local más robusto.

Tabla comparativa: Posit Cloud vs R local

Característica	Posit Cloud	R local (instalado en PC)
Instalación	No requiere instalación	Requiere instalación de R y RStudio
Acceso	Desde cualquier dispositivo con internet	Solo desde la computadora instalada
Conexión a internet	Obligatoria	No obligatoria
Recursos computacionales	Limitados (según el plan)	Dependen del hardware de la PC
Rendimiento	Adecuado para proyectos pequeños/medianos	Mejor para proyectos grandes o pesados
Trabajo colaborativo	Muy sencillo (compartir proyectos online)	Requiere herramientas externas (Git)
Actualizaciones	Automáticas	Manuales
Ideal para	Aprendizaje y prototipado	Desarrollo profesional y análisis intensivo

Data set en R

¿Qué es un dataset? Es una colección organizada de información, generalmente estructurada en forma de tabla para poder analizarla fácilmente.

Tipos de datasets en R:

Internos → vienen incluidos en R o paquetes Ej: `iris`, `mtcars` **Externos** → archivos que importás Ej: CSV, Excel, TXT

Datasets internos -Ya están disponibles sin descargar -Se usan para aprender y practicar **Comandos básicos:**

```
data()      # lista todos los datasets
data(iris)  # carga uno
```

Datasets externos -Se cargan desde archivos

Ejemplos:

```
read.csv("archivo.csv")
read.table("archivo.txt")
```

```
head(datos)      # primeras filas
summary(datos)   # resumen estadístico
str(datos)       # estructura
```

Exploración de datos

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que se le asigne. Se coloca plot y entre paréntesis lo que se quiera graficar. Sepando con comas se van agregando los cambios que se le quieran hacer al gráfico, como el título, lo que esta escrito sobre los ejes, etc. Si no sabemos como es el comando para cambiar algun parámetro del gráfico lo podemos buscar escribiendo en la consola signo de pregunta? seguido del comando plot y explicara en detalle el comando y sus variantes. En la consola podemos escribir y haciendo enter visualizar en la pestaña de la derecha como se vería. Si quedó acorde a lo que queríamos lo copiamos y pegamos en el pestaña de la izquierda superior Source, aquí es donde se hacen todos los cambios que se verán reflejados en el documento que estamos haciendo. Lo que se haga en la consola no queda plasmado.

Descarga de documentos

Tenemos distintas opciones para tejer el documento, para usarlas hay que oprimir el triangulo del ovillo de lana *Knit* y veremos como se despliegan distintas opciones: Knit to HTML nos dará una pagina web, Knit to PDF nos permitirá descargar un PDF, Knit to WORD nos permitirá descargar un archivo WORD.

Funciones estadísticas

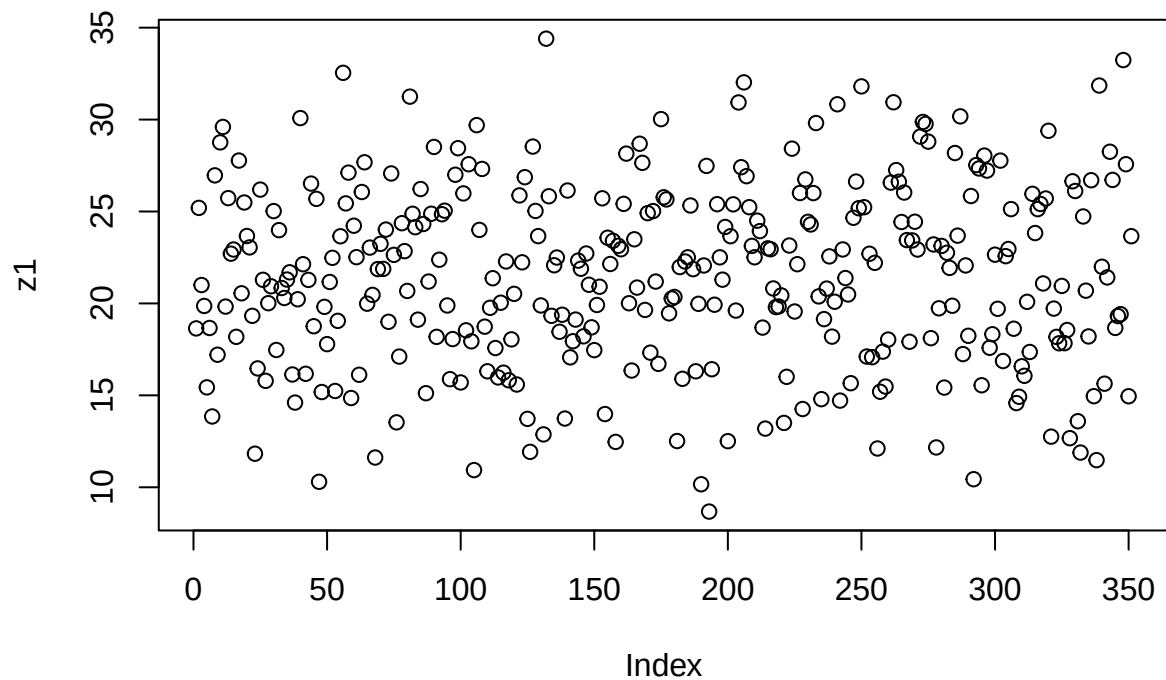
Para utilizar funciones estadísticas debemos colocar un código, por lo tanto, debemos oprimir +C y luego R, se nos despliega el espacio para escribir el código. Allí escribimos el comando rnorm

```
z1<-rnorm(351,22,5)
z1
```

```
## [1] 18.641282 25.197764 21.001087 19.862902 15.434418 18.664287 13.851601
## [8] 26.965530 17.209759 28.760288 29.599141 19.828231 25.728032 22.706479
## [15] 22.932042 18.182708 27.771296 20.551210 25.490304 23.667591 23.048115
## [22] 19.325695 11.825694 16.465763 26.190886 21.285946 15.789025 20.013399
## [29] 20.933690 25.022633 17.468050 23.985726 20.838662 20.295326 21.299071
## [36] 21.693240 16.135482 14.607659 20.226581 30.080761 22.132658 16.172295
## [43] 21.279748 26.518907 18.758923 25.687588 10.298041 15.180073 19.813225
## [50] 17.785198 21.165888 22.480564 15.238140 19.051521 23.653266 32.541954
## [57] 25.440852 27.114825 14.853509 24.226289 22.516322 16.119660 26.060292
## [64] 27.676941 19.985272 23.037531 20.465078 11.618821 21.868605 23.242044
## [71] 21.878003 24.016468 19.000374 27.070333 22.641498 13.531748 17.109532
## [78] 24.364627 22.844725 20.679444 31.247429 24.875681 24.146024 19.116601
## [85] 26.223036 24.319361 15.116789 21.189637 24.878610 28.511932 18.181129
## [92] 22.377321 24.850546 25.045329 19.888603 15.880988 18.060915 26.999987
```

```
## [99] 28.443218 15.701203 25.984058 18.537238 27.571393 17.939424 10.935608
## [106] 29.696471 23.999668 27.314999 18.735787 16.308363 19.761782 21.370934
## [113] 17.573794 15.974354 20.039398 16.235261 22.283770 15.820131 18.044627
## [120] 20.521142 15.588974 25.874154 22.235471 26.866042 13.721530 11.927116
## [127] 28.531384 25.030668 23.661397 19.890014 12.876142 34.399910 25.829591
## [134] 19.326140 22.076342 22.496360 18.459090 19.384738 13.739123 26.141986
## [141] 17.065388 17.942496 19.120964 22.328652 21.886187 18.211446 22.715686
## [148] 21.016164 18.699617 17.460143 19.915119 20.908287 25.718614 13.981785
## [155] 23.572874 22.144890 23.406171 12.467471 23.135176 22.946548 25.415941
## [162] 28.148537 20.010042 16.351272 23.488740 20.861835 28.685482 27.646840
## [169] 19.646541 24.909595 17.322474 25.023689 21.188230 16.712956 30.023960
## [176] 25.772383 25.657579 19.460860 20.267815 20.353491 12.516601 21.976583
## [183] 15.900294 22.280171 22.508132 25.326798 21.863347 16.309380 19.978410
## [190] 10.164741 22.065034 27.479704 8.678598 16.421264 19.929840 25.398070
## [197] 22.506324 21.292545 24.158089 12.507809 23.657349 25.389914 19.612188
## [204] 30.931575 27.409035 32.022506 26.922135 25.235869 23.128512 22.509952
## [211] 24.502518 23.934208 18.685818 13.191354 22.996391 22.947474 20.813658
## [218] 19.799972 19.833902 20.435604 13.500235 16.007790 23.149475 28.419217
## [225] 19.560667 22.133218 26.005704 14.255850 26.746180 24.438423 24.294451
## [232] 26.001664 29.811812 20.382371 14.793157 19.146875 20.807376 22.558802
## [239] 18.194755 20.091977 30.832531 14.714562 22.936457 21.379873 20.488111
## [246] 15.660732 24.658737 26.628445 25.177961 31.805162 25.233716 17.110648
## [253] 22.702352 17.075960 22.207378 12.109551 15.196576 17.378912 15.477543
## [260] 18.033110 26.570189 30.940912 27.252151 26.618405 24.428455 26.031727
## [267] 23.444912 17.915310 23.427801 24.443835 22.931921 29.072953 29.872405
## [274] 29.762127 28.800851 18.109904 23.208945 12.166919 19.735377 23.127170
## [281] 15.422680 22.757371 21.933509 19.873575 28.181477 23.685988 30.174628
## [288] 17.244982 22.065325 18.239776 25.840746 10.439566 27.524857 27.339296
## [295] 15.548338 28.042065 27.227145 17.588599 18.323453 22.652416 19.708502
## [302] 27.769643 16.866697 22.571198 22.959780 25.130236 18.623136 14.579671
## [309] 14.923525 16.579601 16.063605 20.086670 17.358482 25.960976 23.827639
## [316] 25.129655 25.408885 21.087048 25.707221 29.390059 12.753805 19.719559
## [323] 18.180160 17.831546 20.952183 17.837325 18.554749 12.670390 26.643379
## [330] 26.115618 13.592855 11.888020 24.730064 20.695653 18.205151 26.701262
## [337] 14.954509 11.475651 31.851720 21.994999 15.635156 21.414051 28.251192
## [344] 26.719088 18.668073 19.312259 19.415193 33.236700 27.571238 14.951263
## [351] 23.656872
```

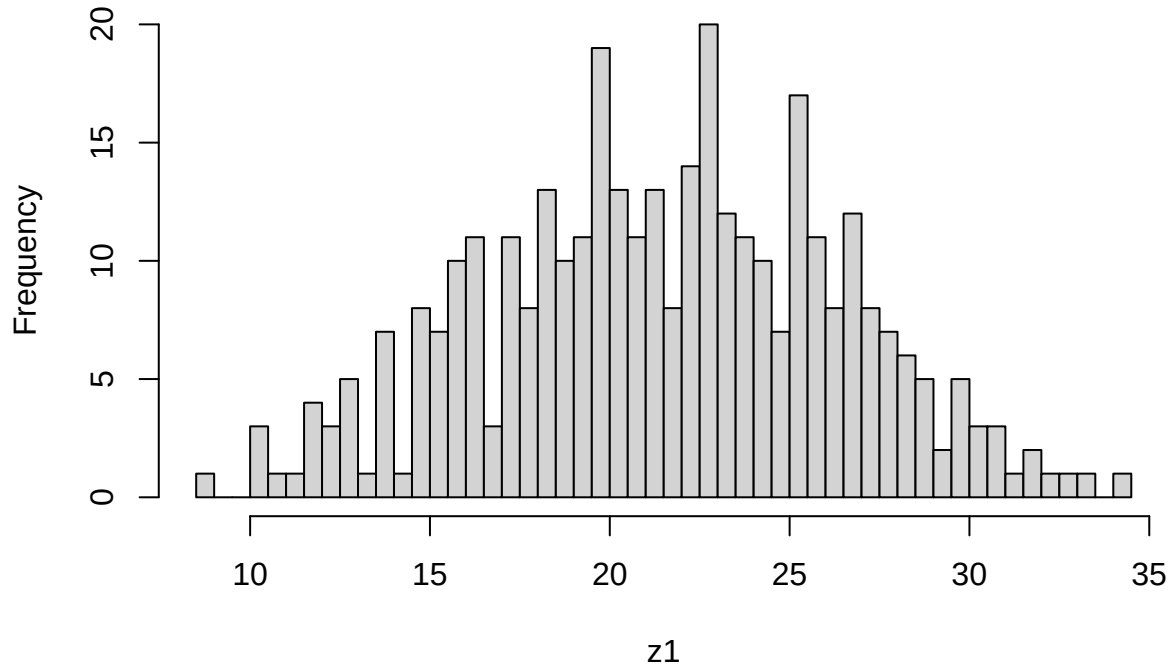
```
plot(z1)
```



un comando para hacer un histograma.

```
hist(z1, main="Histograma de edades", breaks=50)
```

Histograma de edades



density es un comando para hacer un grafico similar a una campana de Gauss pero no es perfecto.

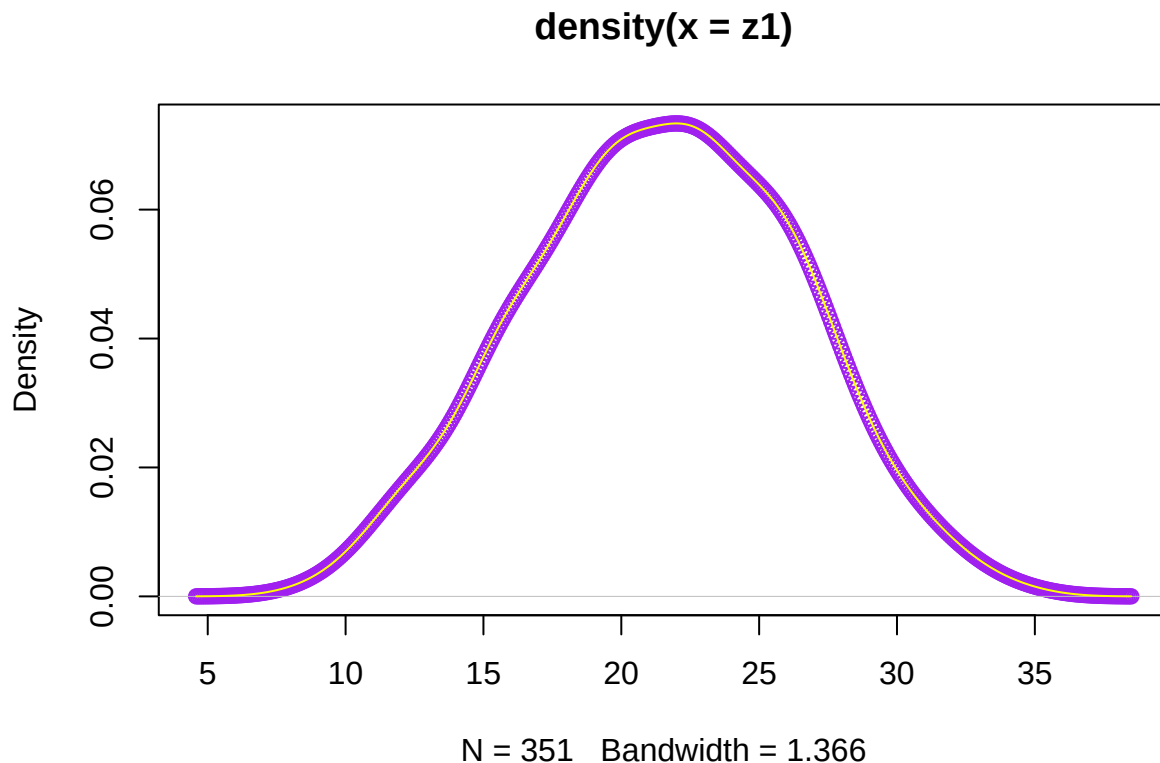
```
density(z1)
```

```
##
```

```
## Call:
```

```
## density.default(x = z1)
##
## Data: z1 (351 obs.); Bandwidth 'bw' = 1.366
##
##      x              y
## Min.   : 4.581   Min.   :9.718e-06
## 1st Qu.:13.060   1st Qu.:2.810e-03
## Median :21.539   Median :2.099e-02
## Mean   :21.539   Mean    :2.943e-02
## 3rd Qu.:30.019   3rd Qu.:5.681e-02
## Max.   :38.498   Max.    :7.337e-02
```

```
plot(density(z1),type="p",col="purple")
lines(density(z1),col="yellow")
```



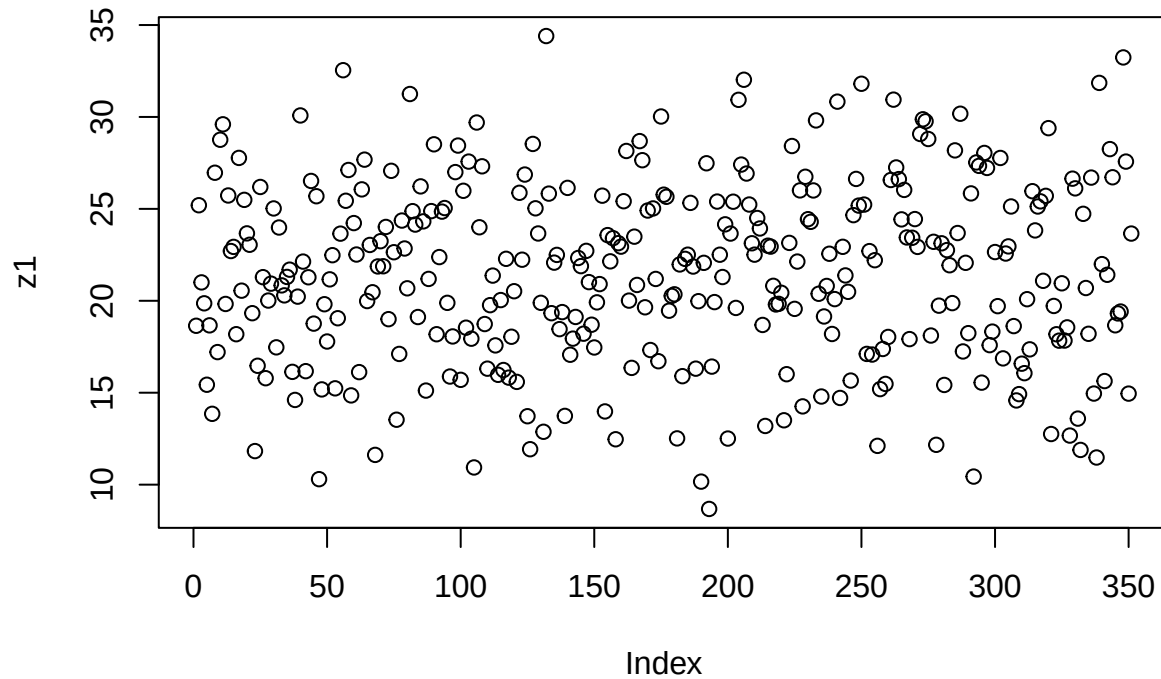
Normal {stats} R Documentation

The Normal Distribution

Description Density, distribution function, quantile function and random generation for the normal distribution with mean equal to mean and standard deviation equal to sd.

Usage `dnorm(x, mean = 0, sd = 1, log = FALSE)` `pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `qnorm(p, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `rnorm(n, mean = 0, sd = 1)`
Arguments `x`, `q` vector of quantiles. `p` vector of probabilities. `n` number of observations. If `length(n) > 1`, the length is taken to be the number required.

```
plot(z1)
```



```
w1<-length(z1)
```

```
w1
```

```
## [1] 351
```

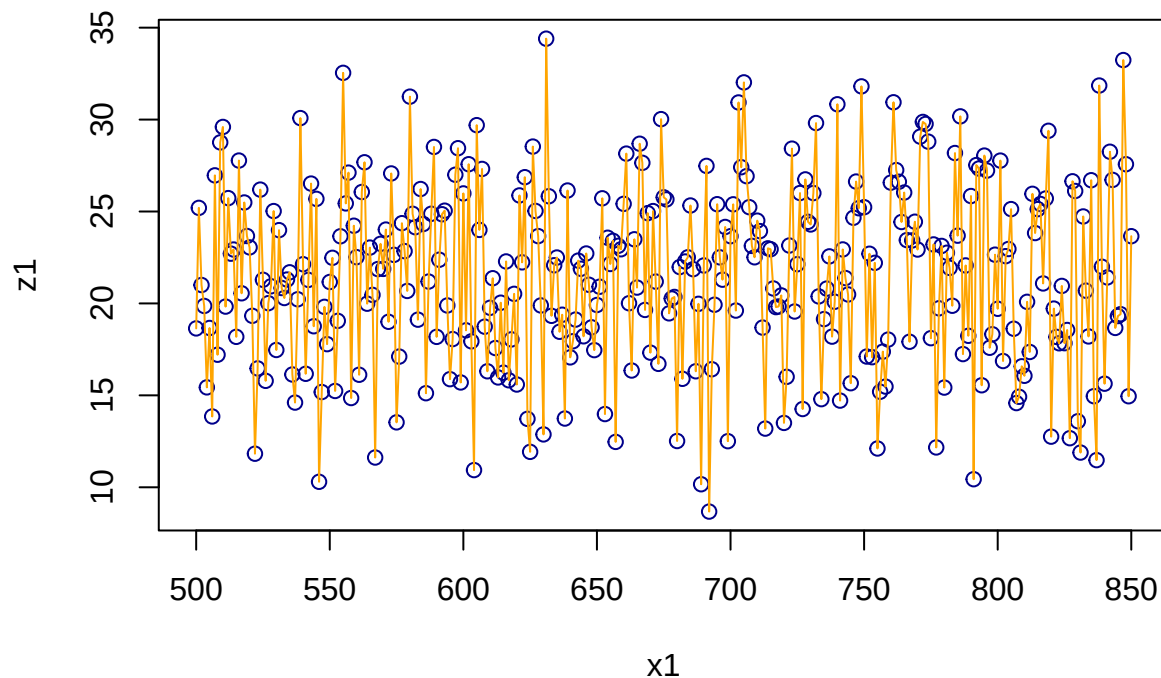
```
x1<-(500:850)
```

```
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849 850
```

```
plot(x1, z1, col="darkblue")
```

```
lines(x1, z1, type="l", col="orange")
```



Clase 8 de Abril

Ejercicio 1 de algoritmos

Consigna 1: Las últimas 3 cifras de mi DNI son 067. Crear una variable que tenga ese número.

```
DNI=067
```

Consigna 2: Crear un vector que tenga todos los números desde el 1 hasta el 067.

```
for(i in 1:067)
print(1:i)
```

```
## [1] 1
## [1] 1 2
## [1] 1 2 3
## [1] 1 2 3 4
## [1] 1 2 3 4 5
## [1] 1 2 3 4 5 6
## [1] 1 2 3 4 5 6 7
## [1] 1 2 3 4 5 6 7 8
## [1] 1 2 3 4 5 6 7 8 9
## [1] 1 2 3 4 5 6 7 8 9 10
## [1] 1 2 3 4 5 6 7 8 9 10 11
## [1] 1 2 3 4 5 6 7 8 9 10 11 12
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```


##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21				
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22			
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23		
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26																								
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27																							
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28																						
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29																					
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30																				
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31																			
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32																		
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33																	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34																
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34	35															
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19						


```

secuencia_DNI <- c()
for(i in 1:067){
  secuencia_DNI <- c(secuencia_DNI, i)
}

```

Ejercicio 2

Consigna 3: Calcular la suma de todos los valores del vector `secuencia_DNI`

```

Total<-0
Valor_final<-length(secuencia_DNI)
for(i in 1:Valor_final){
  Total<-Total+i
}
Total

```

```
## [1] 2278
```

```
print(Total)
```

```
## [1] 2278
```

Ejercicio 3

Consigna 4: Repetir el ejercicio anterior pero en Python

```
{python} secuenciaDNI = range(1, 068) # incluye 067 total = sum(secuenciaDNI) print(total)
```

Ejercicio 4

Consigna 5: ¿Cuánto tarda en correr el *Ejercicio 3*?

```
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##      user  system elapsed
## 0.008   0.003   0.011
```

El comando `system.time` sirve para conocer cuantos milisegundo tarda en ejecutarse un bloque de código

Ejercicio 5

Determinar CPU time used

```

require(stats)
system.time(for(i in 1:100) mad(runif(1000)))

```

```
##      user  system elapsed
## 0.011   0.000   0.013
```

```

require(stats)
DNI= 67
secuencia= 1:DNI

total <- 0
Valor_final <- length(secuencia)

for (i in 1:Valor_final){
  total <- total + secuencia[i]
}

```

```

}
print(total)

## [1] 2278

sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time - start_time

## Time difference of 14.01455 secs

```

Ejercicio 6: Comparación de performance con system.time()

```

system.time({
A <- numeric(50000)
for (i in 1:50000) {
A[i] <- i * 2
}
})

##      user  system elapsed
##    0.004   0.000   0.004

system.time({
B <- seq(from = 2, to = 100000, by = 2)
})

##      user  system elapsed
##         0         0         0

```

Biblioteca tictoc

El uso de una biblioteca en R consiste en incorporar un conjunto de funciones adicionales que amplían las capacidades del lenguaje. Estas bibliotecas contienen procedimientos previamente desarrollados que permiten realizar tareas específicas de manera más sencilla y eficiente. Para poder utilizarlas, primero es necesario instalarlas mediante el comando `install.packages()`, lo que descarga el paquete al entorno de trabajo. Luego, se debe cargar con `library()`, paso que habilita sus funciones para ser utilizadas en la sesión actual.

En el caso de la biblioteca `tictoc`, su finalidad es medir el tiempo de ejecución de un bloque de código. Esta incorpora las funciones `tic()` y `toc()`, que actúan como un cronómetro: la primera inicia la medición y la segunda la finaliza mostrando el tiempo transcurrido. Estas funciones son similares a las utilizadas en Octave o Matlab, manteniendo una lógica de uso sencilla y familiar.

Si bien R dispone de herramientas propias para medir tiempos, como `system.time()`, la biblioteca `tictoc` ofrece una alternativa más cómoda y clara, especialmente cuando se desea evaluar y comparar el rendimiento de distintos procedimientos. De este modo, facilita el análisis de eficiencia del código y mejora la organización del trabajo.

```

library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")

```

```
## [1] "...suena el despertador"
toc()
```

Biblioteca rbenchmark

La función `benchmark` del paquete `rbenchmark` se describe en su documentación como un contenedor sencillo alrededor de la función base `system.time()` de R. Esto significa que internamente utiliza el mismo mecanismo para medir el tiempo de ejecución, pero lo presenta de una manera más práctica y organizada para el usuario.

Aunque `system.time()` permite medir cuánto tarda una expresión en ejecutarse, su uso repetido puede volverse poco cómodo cuando se desean comparar varias expresiones o realizar múltiples repeticiones de cada una. En cambio, `benchmark` simplifica este proceso, ya que permite evaluar varias expresiones dentro de una sola llamada, indicando además cuántas repeticiones se desean realizar.

Otra ventaja importante es que los resultados obtenidos se devuelven en forma de data frame, lo que facilita su lectura, comparación y posterior análisis. De este modo, la función no solo automatiza la medición de tiempos, sino que también organiza la información de manera clara y estructurada, aportando mayor comodidad y eficiencia al trabajo de evaluación de rendimiento en R. *## Implementación de una serie Fibonacci o Fibonacci* En matemáticas, la sucesión de Fibonacci es una sucesión infinita de números naturales que comienza de la siguiente manera:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

La característica principal de esta sucesión es que, a partir de los dos primeros términos (0 y 1), cada número se obtiene sumando los dos anteriores.

Los elementos que forman esta sucesión se denominan números de Fibonacci.

Esta sucesión fue introducida en Europa por Leonardo de Pisa, matemático italiano del siglo XIII, más conocido como Fibonacci, en su obra *Liber Abaci*. Aunque ya era conocida en matemáticas orientales, su difusión en Europa se debe a este autor.

La sucesión de Fibonacci posee numerosas aplicaciones en diversas áreas, como la matemática, la informática, la teoría de juegos e incluso en fenómenos naturales y modelos de crecimiento. Su importancia radica tanto en su sencillez como en las múltiples propiedades y relaciones que presenta dentro del estudio de los números.

Consigna 8: ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```
# Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2    # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
  siguiente <- a + b
  a <- b
  b <- siguiente
  serie <- c(serie, b) # agregamos a la serie
  contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
```

```
## Cantidad de iteraciones: 9
```

```
cat("Primer número mayor a 20:", b, "\n")
```

```
## Primer número mayor a 20: 21
```

```
cat("Serie de Fibonacci hasta ese número:\n")
```

```
## Serie de Fibonacci hasta ese número:
```

```
print(serie)
```

```
## [1] 0 1 1 2 3 5 8 13 21
```

Consigna 9: Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```
# Instalar paquete
```

```
if (!require(microbenchmark)) {  
  install.packages("microbenchmark")  
  library(microbenchmark)  
}
```

```
## Loading required package: microbenchmark
```

```
# Generar muestra
```

```
set.seed(123)  
n <- 20000  
vector <- sample(1:100000, n, replace = TRUE)
```

```
#Implementación de ordenamiento burbuja
```

```
bubble_sort <- function(x) {  
  n <- length(x)  
  for (i in 1:(n-1)) {  
    for (j in 1:(n-i)) {  
      if (x[j] > x[j+1]) {  
        temp <- x[j]  
        x[j] <- x[j+1]  
        x[j+1] <- temp  
      }  
    }  
  }  
  return(x)  
}
```

```
#Benchmark (OJO: burbuja es muy lento)
```

```
resultado <- microbenchmark(  
  burbuja = bubble_sort(vector),  
  sort_R = sort(vector),  
  times = 5  
)  
print(resultado)
```

```
## Unit: microseconds
```

```
##      expr      min       lq      mean     median      uq  
## burbuja 2.22603e+07 22343751.788 2.242761e+07 22354406.681 22499955.460  
## sort_R  5.59671e+02   565.472  6.843058e+02   698.042    730.212  
##      max neval  
## 22679620.062    5  
##      868.132    5
```

La penitencia de Newton

Algunos historiadores relatan que Isaac Newton, durante su etapa escolar, tuvo un maestro sumamente estricto que exigía absoluta atención en clase. Ante cualquier distracción o falta de disciplina, imponía como castigo una tarea particularmente tediosa: calcular para el día siguiente la suma de todos los números desde 1 hasta un número elegido al azar.

A simple vista, esta consigna implicaba realizar largas sumas parciales, una por una, lo que podía demandar horas de trabajo. El profesor, precavido, ya tenía resueltos estos cálculos y los llevaba anotados en un cuaderno voluminoso, similar en utilidad a una tabla de logaritmos, que transportaba cuidadosamente en su portafolio.

La anécdota cobra relevancia porque, según la tradición, el joven Newton habría encontrado una forma más rápida y elegante de realizar esas sumas, descubriendo un patrón que permitía obtener el resultado sin necesidad de sumar término por término. Más allá de su veracidad histórica, el relato ilustra cómo la observación y el razonamiento pueden transformar una tarea repetitiva en un problema matemático con una solución general.

Consigna 10: Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta $1 \cdot 1010$ y verifica cuál de los dos es más eficiente.

```
#Algoritmo 1: suma con for (iterativo)
```

```
n <- 1010
suma1 <- 0
for (i in 1:n){
  suma1 <- suma1 + i
}
suma1
```

```
## [1] 510555
```

```
#Complejidad: O(n) (suma uno por uno)
```

```
#Algoritmo 2: fórmula matemática (Gauss)
```

```
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
```

```
## [1] 510555
```

```
#Complejidad: O(1) (una sola operación)
```

```
#Comparación de eficiencia: medir tiempos con system.time():
```

```
# Método 1
```

```
system.time({
  suma1 <- 0
  for (i in 1:1010){
    suma1 <- suma1 + i
  }
})
```

```
##      user  system elapsed
```

```
## 0.002 0.000 0.002
```

```
# Método 2
```

```
system.time({
  suma2 <- 1010 * (1010 + 1) / 2
})
```

```
##      user  system elapsed
```

```
##      0      0      0
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta $1 \cdot 10^{10}$. El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente

```
##Matriz Inversa
##Definir la matriz
A <- matrix(c(4, 7,
              2, 6),
            nrow = 2,
            byrow = TRUE)

##Calcular la inversa
A_inv <- solve(A)

##Mostrar resultado
A_inv

##      [,1] [,2]
## [1,]  0.6 -0.7
## [2,] -0.2  0.4
```

Investigar con los gráficos de violín (de la biblioteca micro como se comporta una computadora usando métodos de ordenamiento “burbuja” y compararlo con el método sort nativo de R .

Medir la performance del método kmean de agrupamiento. Realizar una introducción teórica antes de medir con gráfico de violín.

```
library(microbenchmark)
library(ggplot2)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##      filter, lag

## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union

# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
}
```



```

    }
    return(x)
  }

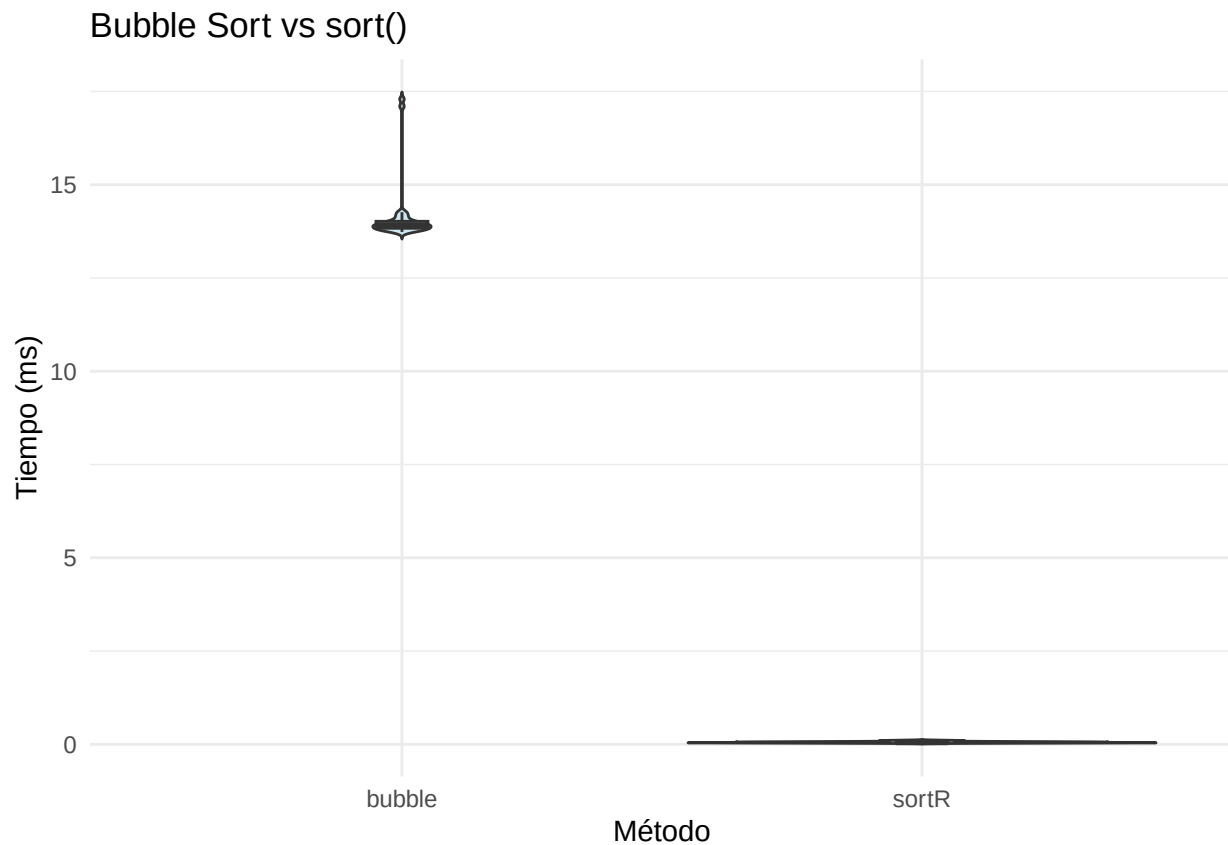
# Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)

# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 30
)

# Convertir a ms
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
  geom_violin(trim = FALSE, fill = "lightblue", alpha = 0.6) +
  geom_boxplot(width = 0.1, outlier.shape = NA) +
  labs(title = "Bubble Sort vs sort()",
       x = "Método",
       y = "Tiempo (ms)") +
  theme_minimal()

```



```
library(microbenchmark)
library(ggplot2)
library(dplyr)

# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)

# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
```

```

times = 30
)

# Convertir a ms
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
  geom_violin(trim = FALSE, fill = "blue", alpha = 0.6) +
  geom_boxplot(width = 0.1, outlier.shape = NA) +
  labs(title = "Bubble Sort vs sort()",
       x = "Método",
       y = "Tiempo (ms)") +
  theme_minimal()

```

